Firjan SENAI

Situação de Aprendizagem

SITUAÇÃO DE APRENDIZAGEM	
Segmento:	Técnico
Unidade (s) Curricular (es):	Desenvolvimento de Sistemas
Curso (s):	Desenvolvimento de Sistemas
Instrutor (a):	Vitor Hugo Almeida de Sousa

ESTRATÉGIA DE APRENDIZAGEM DESAFIADORA

[x] Situação-problema [ ] Estudo do Caso [ ] Projeto [ ] Pesquisa Aplicada

DESCRIÇÃO DA ESTRATÉGIA DE APRENDIZAGEM DESAFIADORA

Contextualização: Encerramento e Consulta de Ordens Abertas

A equipe de manutenção já registra as solicitações e identifica o equipamento e o momento da abertura. Agora, precisa registrar a solução aplicada e quando o atendimento terminou, além de consultar as solicitações pendentes.

Você acrescentará o encerramento e a consulta de ordens abertas ao mesmo projeto, preservando o histórico e praticando DTO, validação, datas e regras de negócio.

DESAFIO

Etapa 1 — Preparar o projeto e o Git

• Execute a API e consulte GET /ordens-servico. Separe o ID de uma ordem existente para os testes.

• Confira git status e registre alterações pendentes. Parta da main com a abertura de OS concluída.

• Utilize o DEV_SIS_12 como referência para devolução e consulta de empréstimos ativos.

Execute:

```
git switch main
git switch -c encerramento-os
git push -u origin encerramento-os
```

Etapa 2 — Complementar o Model OrdemServico

• Em OrdemServico.java, mantenha id, descricao, dataAbertura e equipamento.

• Acrescente LocalDateTime dataEncerramento — data e hora em que o atendimento terminou. Utilize java.time.LocalDateTime.

• Acrescente String solucao — descrição do serviço realizado.

• Adicione getters e setters para os dois atributos.

• No cadastro, dataEncerramento e solucao permanecem null. Continue preenchendo apenas dataAbertura com a data e a hora do sistema.

• Uma ordem está aberta quando dataEncerramento é null; está encerrada quando esse campo possui valor.

• Reinicie a aplicação e confira as novas colunas na tabela ordem_servico. Preserve os registros existentes.

Etapa 3 — Criar o DTO de encerramento

• No pacote dto, crie EncerramentoOrdemServicoRequest.java.

• Declare somente String solucao, com @NotBlank e uma mensagem clara, como “Informe a solução realizada”.

• Acrescente construtor vazio, getter e setter.

• O DTO não recebe equipamentoId nem datas. O ID da ordem virá no endereço e a data do encerramento será preenchida pelo servidor.

Etapa 4 — Implementar o encerramento no Service

• Em OrdemServicoService, crie encerrar(Integer id, EncerramentoOrdemServicoRequest dados).

• Busque a ordem pelo ID usando findById(). Se ela não existir, siga o padrão de Optional vazio já utilizado para que o Controller retorne 404.

• Se dataEncerramento já estiver preenchida, interrompa a operação e responda 409 Conflict com uma mensagem indicando que a ordem já foi encerrada. Não substitua a solução nem a data anteriores.

• Se a ordem estiver aberta, copie a solução do DTO, preencha dataEncerramento e salve a entidade. Retorne a ordem atualizada.

Dica para registrar data e hora:

```
ordem.setSolucao(dados.getSolucao());
ordem.setDataEncerramento(LocalDateTime.now());
```

• Preserve descricao, equipamento e dataAbertura. Registre o encerramento uma única vez.

Dica para a regra de negócio

• Consulte o tratamento de 409 Conflict utilizado no DEV_SIS_12. Se seguir esse padrão, use ResponseStatusException com HttpStatus.CONFLICT e uma mensagem clara.

• Essa regra fica no Service, pois compara a solicitação com o estado da ordem no banco. @NotBlank verifica somente o preenchimento da solução.

Etapa 5 — Disponibilizar a rota PATCH

• Em OrdemServicoController, crie PATCH /ordens-servico/{id}/encerramento. Use @PatchMapping("/{id}/encerramento").

• Receba o ID com @PathVariable e o DTO com @Valid @RequestBody EncerramentoOrdemServicoRequest.

• Encaminhe os dados ao Service. Retorne 200 OK com a ordem atualizada ou 404 Not Found quando a ordem não existir.

• O ValidacaoExceptionHandler já criado deve devolver a mensagem de solucao com 400 Bad Request. A regra do Service impede um segundo encerramento com 409.

No Postman, selecione PATCH e informe:

http://localhost:8080/ordens-servico/1/encerramento

Em Body → raw → JSON, envie:

```
{
  "solucao": "Realizada a substituição do sensor e o teste de funcionamento."
}
```

Substitua 1 pelo ID real de uma ordem aberta. Não envie dataEncerramento.

Etapa 6 — Consultar somente as ordens abertas

• No OrdemServicoRepository, declare List<OrdemServico> findByDataEncerramentoIsNull(); e importe java.util.List.

• Dica: findBy solicita a busca; DataEncerramento indica o atributo; IsNull seleciona registros sem encerramento.

• No Service, crie listarAbertas(), utilizando essa consulta.

• No Controller, crie GET /ordens-servico/abertas com 200 OK. Sem pendências, retorne uma lista vazia.

• Mantenha GET /ordens-servico retornando todas as ordens, abertas e encerradas.

Etapa 7 — Testar os cenários no Postman

• Consulte as ordens abertas e anote os IDs. Encerre uma delas com solução válida: espere 200 e confira solucao e dataEncerramento com data e hora.

• Consulte a mesma ordem por ID: a solução e as datas devem permanecer iguais. Confirme que ela continua na lista geral e saiu da lista de abertas.

• Repita o encerramento com outra solução: espere 409 e confirme pelo GET que a primeira solução e a data foram preservadas.

• Use um ID inexistente com solução válida: espere 404.

• Em uma ordem aberta, teste solucao vazia, somente espaços, null e campo omitido: espere 400 com a mensagem do campo. Confirme que ela continua aberta, sem solução gravada.

• Salve cada requisição e descreva o resultado esperado. Não fixe um ID sem consultar os registros reais.

Etapa 8 — Documentar e concluir no Git

• Atualize a pasta “Ordens de Serviço” da collection com PATCH de encerramento e GET de abertas.

• Confira se o cadastro e as consultas anteriores continuam funcionando.

• Registre e envie as alterações:

```
git add .
git commit -m "feat: consultar ordens abertas e testar encerramento"
git push
```

• Após todos os testes passarem, conclua:

```
git switch main
git merge encerramento-os
git push origin main
```

• Mantenha a branch encerramento-os como registro.

RESULTADOS E ENTREGAS ESPERADOS
• Entregue o link do mesmo repositório, com os commits da atividade e a branch encerramento-os. Exporte a collection atualizada. • Demonstre encerramento válido, solução inválida, ordem inexistente e tentativa de encerrar novamente. • Confira: a primeira solução e a data são preservadas; ordens encerradas ficam no histórico; somente ordens sem dataEncerramento aparecem na lista de abertas. • Explique por que a validação da solução fica no DTO e a verificação de ordem já encerrada fica no Service.

Material de Consumo:	Não se aplica
Ferramentas:	IDE; MySQL Workbench; Postman; Git e GitHub; Spring Boot
EPIs:	Não se aplica
EPCs:	Não se aplica
Equipamentos:	Computador com Java/JDK, MySQL e acesso às dependências do projeto
Instrumentos:	Não se aplica